PROJET OPENVPN

Infrastructure VPN & Certificats PKI

BTS SIO — Option SISR

Linux / Debian	OpenVPN	EasyRSA (PKI)	SSL / TLS

Auteur
Ilan Benamou
BTS SIO — Option SISR

Projet
OpenVPN + PKI / EasyRSA
Année 2024 – 2025

Table des matières

I.	**Présentation du projet et contexte**	3
II.	**Architecture technique**	3
III.	**Mise en place de l'autorité de certification (PKI)**	4
	A. Installation d'EasyRSA	4
	B. Initialisation du PKI et création du CA	4
	C. Génération des certificats serveur	5
	D. Génération des certificats clients	5
IV.	**Configuration du serveur OpenVPN**	6
	A. Installation d'OpenVPN	6
	B. Configuration server.conf	6
	C. Activation du routage et iptables (NAT)	7
V.	**Configuration des clients VPN**	8
	A. Fichier de configuration .ovpn	8
	B. Test de connexion	8
VI.	**Compétences acquises et conclusion**	9

I. Présentation du projet et contexte

Avec l'essor du télétravail, sécuriser les accès distants est devenu une priorité absolue. Ce projet vise à déployer une infrastructure **VPN (Virtual Private Network)** complète pour permettre à des collaborateurs nomades d'accéder aux ressources internes de l'entreprise de manière chiffrée.

Objectif : Mettre en œuvre le protocole OpenVPN et gérer une infrastructure à clés publiques (PKI) pour authentifier les utilisateurs via des certificats numériques.

Récapitulatif de l'environnement technique

Composant	Détail
Système d'exploitation	Debian 11 (Bullseye) — serveur Linux en ligne de commande
Logiciel VPN	OpenVPN (protocole SSL/TLS)
Gestion PKI	EasyRSA 3.x — Autorité de Certification locale
Protocole de transport	UDP (port 1194 par défaut)
Chiffrement	AES-256-CBC — clé de 256 bits
Authentification	Certificats numériques X.509 (CA, serveur, clients)
Routage	IP Forwarding + iptables NAT (MASQUERADE)
Réseau VPN	Tunnel 10.8.0.0/24
Réseau LAN interne	Accessible via le tunnel VPN

II. Architecture technique

L'infrastructure OpenVPN repose sur quatre composants interconnectés qui assurent ensemble la sécurité, le routage et la gestion des identités.

Serveur OpenVPN

Installation et configuration du démon OpenVPN (fichier server.conf) : choix du port, du protocole UDP et du chiffrement AES-256.

Gestion PKI (EasyRSA)

Création d'une Autorité de Certification (CA) avec EasyRSA. Génération et signature des certificats serveur et clients.

Client VPN

Déploiement du fichier de configuration (.ovpn) sur les postes clients et test de la connexion tunnelisée.

Routage & IP

Configuration de l'IP Forwarding sur le serveur Linux et règles iptables (NAT) pour permettre l'accès au LAN.

Le flux d'une connexion VPN se déroule comme suit : le client envoie une requête chiffrée au serveur OpenVPN via UDP/1194. Le serveur vérifie le certificat client signé par le CA. Si valide, un tunnel TLS est établi et le client reçoit une IP dans le réseau 10.8.0.0/24. Grâce aux règles iptables NAT, le client peut ensuite accéder aux ressources du LAN interne.

III. Mise en place de l'Autorité de Certification (PKI)

Le PKI (Public Key Infrastructure) est le cœur de la sécurité OpenVPN. Il permet d'émettre, de signer et de révoquer des certificats numériques. EasyRSA 3 est l'outil utilisé pour gérer cette infrastructure.

A. Installation d'EasyRSA

Sur le serveur Debian, installer OpenVPN et EasyRSA :

```
apt update && apt upgrade -y
apt install openvpn easy-rsa -y
```

Créer le répertoire de travail pour le PKI :

```
make-cadir /etc/openvpn/easy-rsa
cd /etc/openvpn/easy-rsa
```

> *Le répertoire /etc/openvpn/easy-rsa contient tous les scripts EasyRSA et sera le dossier de travail pour toutes les opérations PKI (création CA, certificats, clés).*

B. Initialisation du PKI et création du CA

Initialiser le PKI, puis créer l'Autorité de Certification racine :

```
./easyrsa init-pki
./easyrsa build-ca nopass
```

EasyRSA demande le Common Name du CA. Renseigner un nom explicite (ex : OpenVPN-CA). Cette commande génère deux fichiers essentiels :

Fichier	Rôle
`pki/ca.crt`	Certificat public du CA — distribué à tous les clients
`pki/private/ca.key`	Clé privée du CA — à conserver secrète, ne jamais distribuer

C. Génération des certificats serveur

Générer la demande de certificat et la clé privée du serveur, puis la signer avec le CA :

```
./easyrsa gen-req server nopass
./easyrsa sign-req server server
```

Générer également les paramètres Diffie-Hellman (nécessaires pour l'échange de clés TLS) et la clé TLS-Auth :

```
./easyrsa gen-dh
openvpn --genkey --secret ta.key
```

Copier tous les fichiers nécessaires dans le répertoire de configuration OpenVPN :

```
cp pki/ca.crt /etc/openvpn/
cp pki/issued/server.crt /etc/openvpn/
cp pki/private/server.key /etc/openvpn/
cp pki/dh.pem /etc/openvpn/
cp ta.key /etc/openvpn/
```

> *Les paramètres DH (dh.pem) sont calculés une seule fois et peuvent prendre quelques minutes à générer. Ils garantissent la confidentialité persistante (Perfect Forward Secrecy).*

D. Génération des certificats clients

Pour chaque utilisateur client, générer un certificat dédié (exemple pour l'utilisateur 'client1') :

```
./easyrsa gen-req client1 nopass
./easyrsa sign-req client client1
```

Les fichiers générés à transmettre au client sont :

Fichier	Usage
`ca.crt`	Certificat de l'autorité de certification (racine de confiance)
`pki/issued/client1.crt`	Certificat public du client, signé par le CA
`pki/private/client1.key`	Clé privée du client (à protéger)
`ta.key`	Clé TLS-Auth pour l'authentification supplémentaire HMAC

IV. Configuration du serveur OpenVPN

A. Installation d'OpenVPN

OpenVPN est installé depuis les dépôts Debian (déjà fait à l'étape précédente). Vérifier que le service est disponible :

```
openvpn --version
systemctl status openvpn
```

B. Fichier de configuration server.conf

Créer le fichier de configuration principal du serveur dans /etc/openvpn/ :

```
nano /etc/openvpn/server.conf
```

Contenu du fichier server.conf :

```
# Protocole et port
port 1194
proto udp
dev tun

# Certificats et clés
ca   /etc/openvpn/ca.crt
cert /etc/openvpn/server.crt
key  /etc/openvpn/server.key
dh   /etc/openvpn/dh.pem
tls-auth /etc/openvpn/ta.key 0

# Réseau VPN
server 10.8.0.0 255.255.255.0
ifconfig-pool-persist /var/log/openvpn/ipp.txt

# Routage LAN — pousser les routes vers les clients
push "redirect-gateway def1 bypass-dhcp"
push "dhcp-option DNS 8.8.8.8"

# Sécurité et chiffrement
cipher AES-256-CBC
auth SHA256
tls-version-min 1.2

# Persistance et logs
keepalive 10 120
persist-key
persist-tun
status /var/log/openvpn/openvpn-status.log
verb 3
```

> *La directive 'server 10.8.0.0 255.255.255.0' crée le réseau virtuel du tunnel. Le serveur obtient l'IP 10.8.0.1, les clients reçoivent les adresses suivantes (10.8.0.2, etc.).*

Activer et démarrer le service OpenVPN :

```
systemctl enable openvpn@server
systemctl start  openvpn@server
systemctl status openvpn@server
```

C. Activation du routage et règles iptables (NAT)

Pour que les clients VPN puissent accéder au LAN interne, il faut activer l'IP Forwarding (permettre au serveur de relayer les paquets) et configurer le NAT avec iptables.

1. Activer l'IP Forwarding

Éditer le fichier sysctl pour activer le forwarding de façon permanente :

```
nano /etc/sysctl.conf
# Décommenter ou ajouter la ligne suivante :
net.ipv4.ip_forward = 1
```

Appliquer immédiatement sans redémarrage :

```
sysctl -p
# Vérification
cat /proc/sys/net/ipv4/ip_forward   # doit afficher : 1
```

2. Configurer iptables (NAT / MASQUERADE)

Ajouter la règle de masquerade pour que le trafic VPN sorte avec l'IP du serveur (remplacer eth0 par l'interface réseau réelle) :

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
```

Rendre cette règle persistante après redémarrage :

```
apt install iptables-persistent -y
netfilter-persistent save
netfilter-persistent reload
```

> *Sans la règle MASQUERADE, les clients VPN peuvent joindre le serveur mais pas le LAN interne. La règle NAT traduit l'adresse source 10.8.x.x en adresse IP du serveur pour que les machines du LAN puissent répondre.*

Vérifier les règles iptables actives :

```
iptables -t nat -L -n -v
```

V. Configuration des clients VPN

A. Fichier de configuration .ovpn

Le fichier .ovpn regroupe toute la configuration et les certificats nécessaires au client. Créer le fichier client1.ovpn en intégrant les certificats directement (mode inline) :

```
nano client1.ovpn

client
dev tun
proto udp
remote <IP_DU_SERVEUR> 1194
resolv-retry infinite
nobind
persist-key
persist-tun

# Chiffrement (doit correspondre au serveur)
cipher AES-256-CBC
auth SHA256
tls-version-min 1.2
key-direction 1

<ca>
  [contenu de ca.crt]
</ca>
<cert>
  [contenu de client1.crt]
</cert>
<key>
  [contenu de client1.key]
</key>
<tls-auth>
  [contenu de ta.key]
</tls-auth>
```

> *Le mode 'inline' intègre tous les certificats dans un seul fichier .ovpn, ce qui facilite la distribution aux utilisateurs finaux. Ce fichier doit être transmis de façon sécurisée (SFTP, clé USB chiffrée, etc.).*

B. Test de connexion

Sur le poste client (Linux ou Windows avec OpenVPN GUI), lancer la connexion :

```
# Sur Linux
openvpn --config client1.ovpn

# Vérifier que l'interface tun0 est créée avec une IP 10.8.0.x
ip a show tun0

# Tester la connectivité vers le serveur VPN
```

```
ping 10.8.0.1

# Tester l'accès au LAN interne
ping <IP_MACHINE_LAN>
```

Côté serveur, vérifier les connexions actives :

```
cat /var/log/openvpn/openvpn-status.log
journalctl -u openvpn@server -f
```

> *Une connexion réussie affiche dans les logs : 'client1/IP:PORT MULTI: Learn: 10.8.0.x' et 'Initialization Sequence Completed'. L'interface tun0 du client reçoit une adresse dans 10.8.0.0/24.*

VI. Compétences acquises et conclusion

Ce projet m'a permis de comprendre en profondeur le fonctionnement du tunneling et de la cryptographie asymétrique. Je sais désormais mettre en place un accès distant sécurisé, compétence critique pour la continuité d'activité.

Compétences développées

Infrastructure PKI

Création d'une CA, génération et signature de certificats X.509 avec EasyRSA. Compréhension de la chaîne de confiance.

VPN SSL/TLS

Configuration complète d'OpenVPN (server.conf), choix du protocole UDP, chiffrement AES-256, TLS-Auth.

Réseau Linux (iptables)

Activation de l'IP Forwarding, règles NAT MASQUERADE avec iptables pour l'accès au LAN, persistance des règles.

Déploiement client

Création de fichiers .ovpn inline, déploiement multi-plateforme (Linux/Windows), test et validation de la connexion tunnelisée.

Bilan du projet

Infrastructure OpenVPN opérationnelle avec authentification par certificats PKI.

Tunnel chiffré AES-256 — accès sécurisé au LAN interne depuis n'importe quel réseau.

Compétence directement applicable en entreprise pour la continuité d'activité et le télétravail sécurisé.